

Lehome challenge 2026

Process

1.

data collection

From official

collect by ourself

2.

training

X-VLA

3.

evaluate

Data collection

From official

merge version

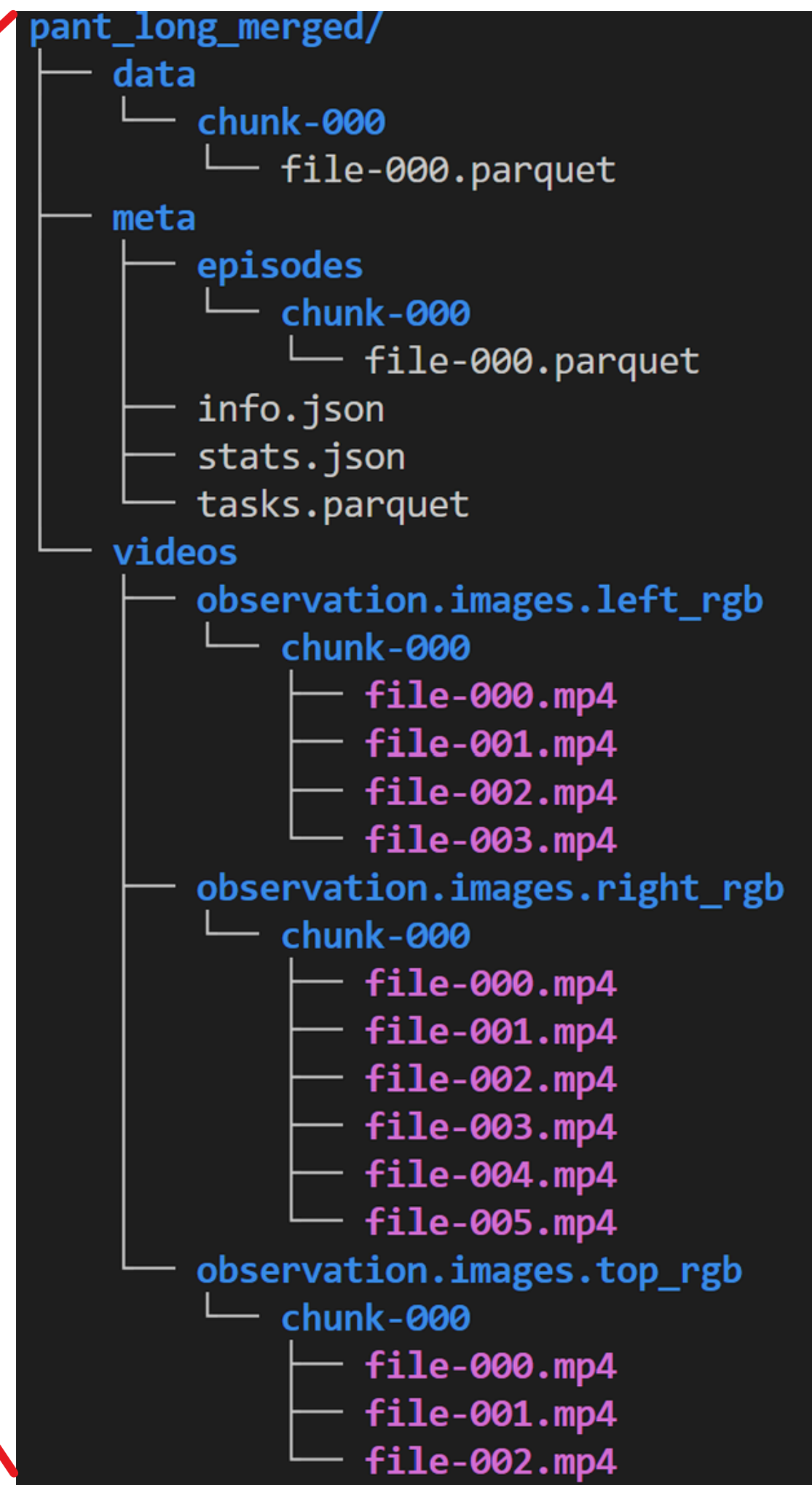
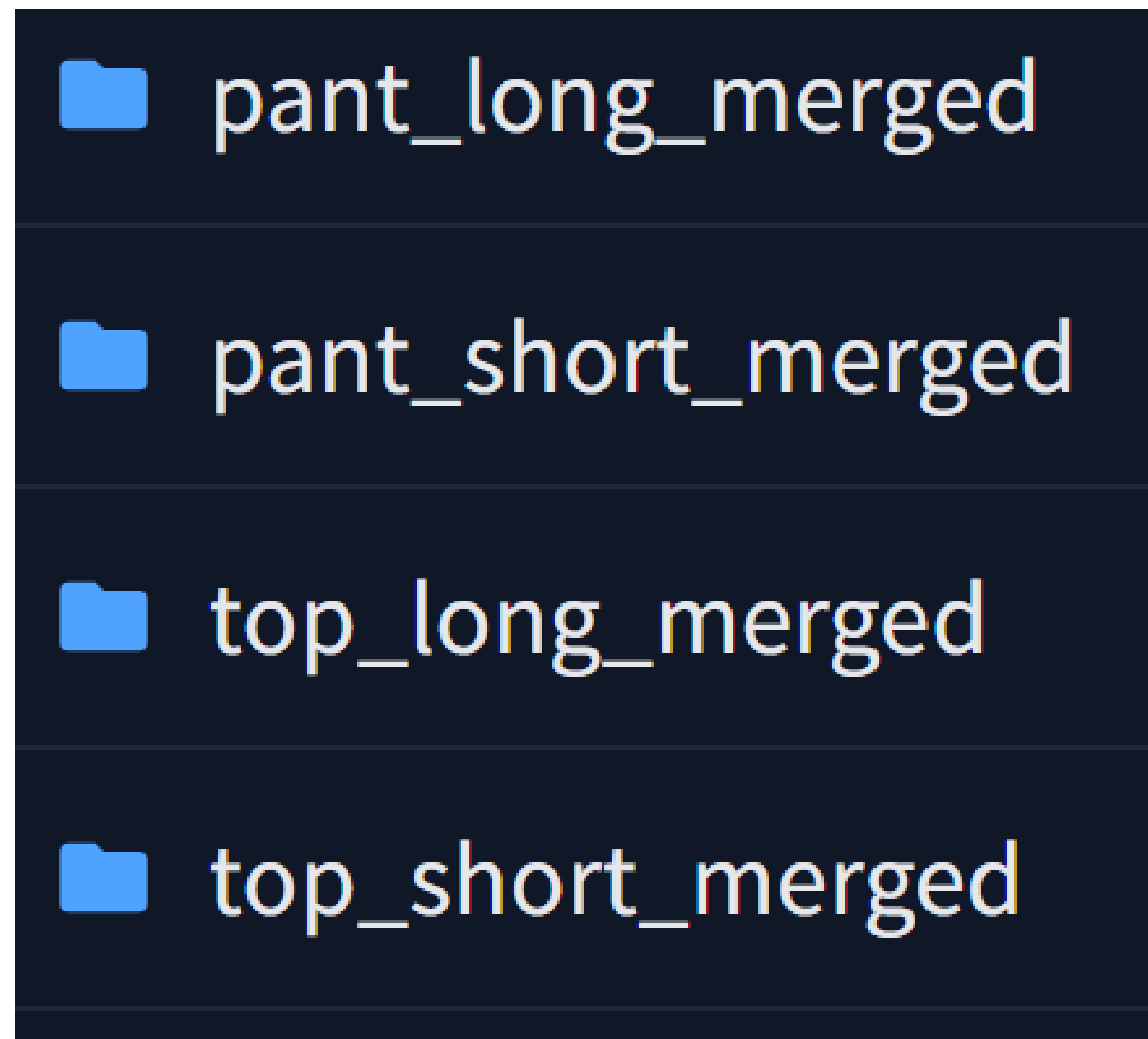
- 📁 pant_long_merged
- 📁 pant_short_merged
- 📁 top_long_merged
- 📁 top_short_merged

none merge version

- 📁 record_pant_long_release_10
- 📁 record_pant_short_release_10
- 📁 record_top_long_release_10
- 📁 record_top_short_release_10

Data collection

merge version



Data collection

none merge version

record_pant_long_release_10

record_pant_short_release_10

record_top_long_release_10

record_top_short_release_10

record_pant_long_release_10

> 001
> 002
> 003
> 004
> 005
> 006
> 007
> 008
> 009
> 010

record_pant_long_release_10/

001

data

chunk-000

file-000.parquet

file-001.parquet

file-002.parquet

file-003.parquet

meta

episodes

chunk-000

file-000.parquet

garment_info.json

info.json

stats.json

tasks.parquet

videos

observation.images.left_rgb

chunk-000

file-000.mp4

observation.images.right_rgb

chunk-000

file-000.mp4

observation.images.top_rgb

chunk-000

file-000.mp4

Data collection

From official

merge version

none merge version



Direct use



observation
.top_depth



Data collection

collect by ourself

keyboard

Single-arm: WASD (position), QE (Z-axis), Arrow keys (orientation), Space/Shift (gripper)

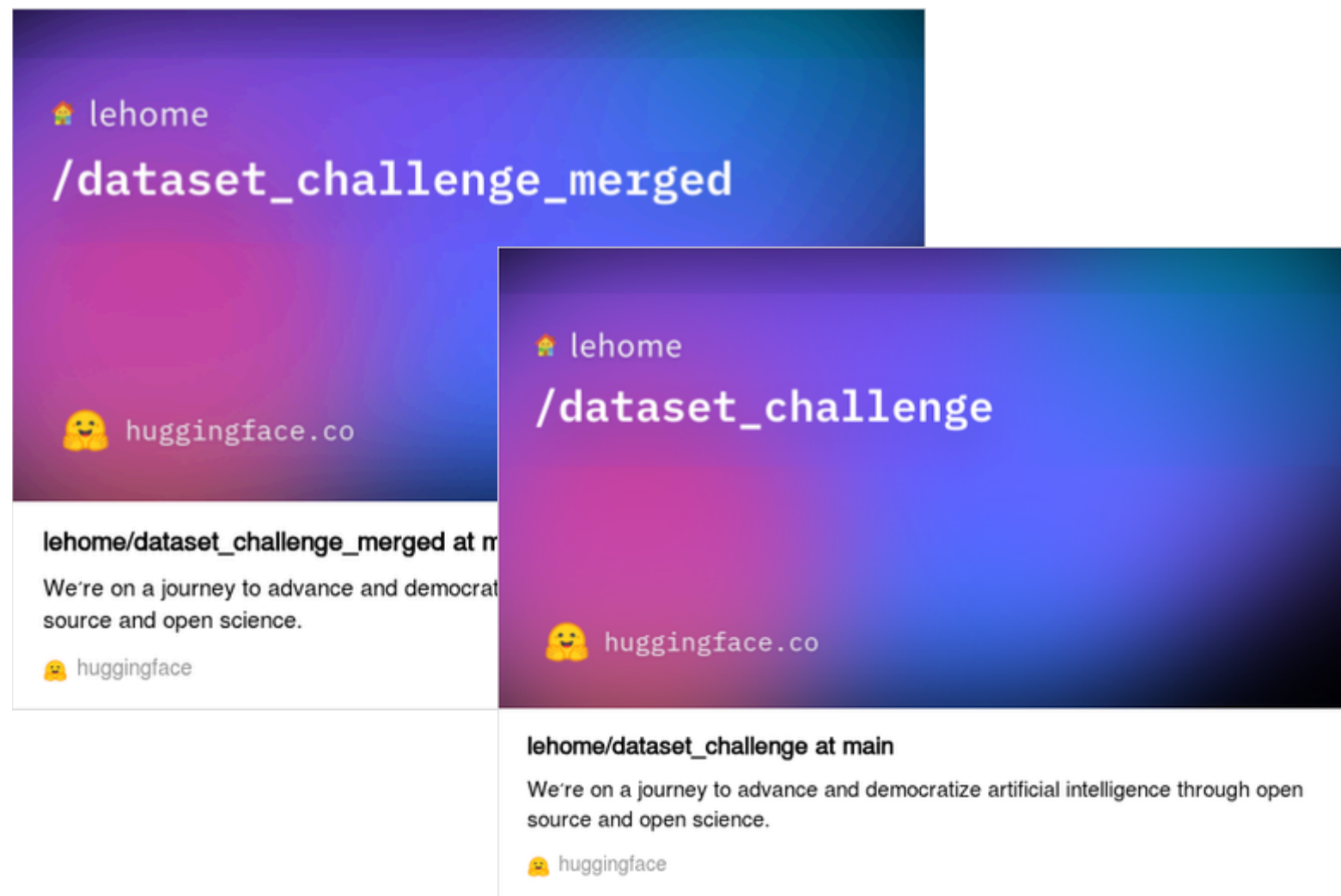
Dual-arm: Letter keys (T/G, Y/H, U/J, I/K, O/L, Q/A) for left arm, Number keys (1/2, 3/4, 5/6, 7/8, 9/0, [/] or -/+) for right arm

SO101 Leader Arms

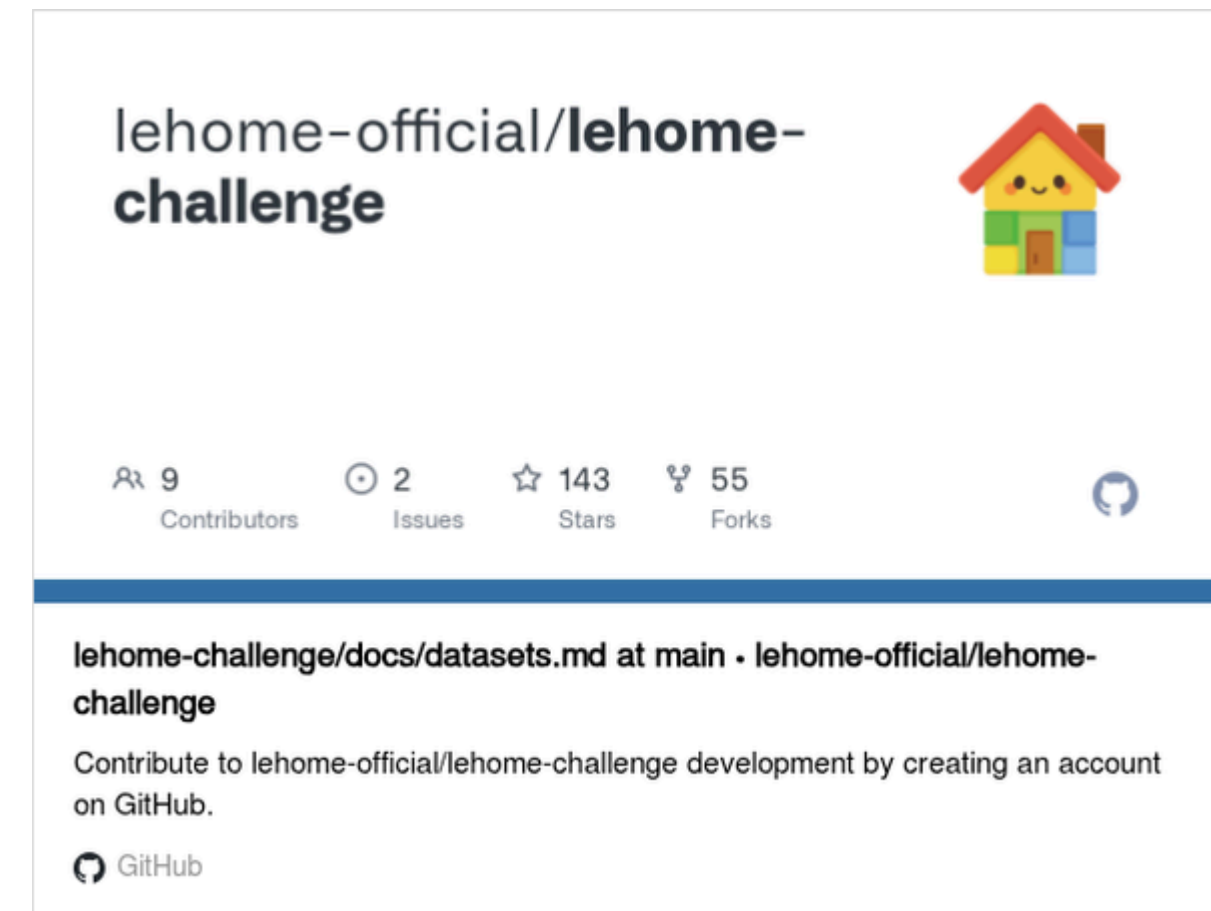
Directly move the physical Leader Arms; the simulated robots follow in real-time

Data collection

From official



collect by us



training

Using pre-configured

Train a policy using one of the pre-configured training files:

```
lerobot-train --config_path=configs/train_act.yaml
```

Available config files:

- `configs/train_act.yaml` - ACT policy
- `configs/train_dp.yaml` - Diffusion Policy
- `configs/train_smolvla.yaml` - SmolVLA policy

Bring our own policy

LeRobot Docs

Bring Your Own Policies

training

Using pre-configured

Train a policy using one of the pre-configured training files:

💡 **Note:** LeRobot supports additional policies (π_0 , $\pi_{0.5}$, GR00T, X-VLA), but configuration files for these are not provided in this repository. You can create custom configuration files following the [LeRobot documentation](#) or use the above three baseline policies.

💡 **注意：** LeRobot 支援其他策略 (π_0 、 $\pi_{0.5}$ 、GR00T、X-VLA)，但本倉庫未提供這些策略的設定檔。您可以按照 [LeRobot 文件](#) 建立自訂設定文件，或使用上述三種基本策略。

- `configs/train_dp.yaml` - Diffusion Policy
- `configs/train_smolvla.yaml` - SmolVLA policy

training

Bring our own policy

LeRobot Docs

Bring Your Own Policies



click it

training

Bring our own policy

```
lerobot_policy_my_custom_policy/  
├─ pyproject.toml  
└─ src/  
    └─ lerobot_policy_my_custom_policy/  
        ├─ __init__.py  
        ├─ configuration_my_custom_policy.py  
        ├─ modeling_my_custom_policy.py  
        └─ processor_my_custom_policy.py
```

train_your_policy.yaml

```
dataset:  
  repo_id: local_dataset_001  
  root: Datasets/record/001  
  
policy:  
  type: my_custom_policy          # Your cu  
  device: cuda  
  
input_features:  
  # Define your input features  
  observation.state:  
    type: STATE  
    shape: [12]  
  observation.images.top_rgb:  
    type: VISUAL  
    shape: [3, 480, 640]
```

training

Using pre-configured

Bring our own policy

lehome-official/**lehome-challenge**



9
Contributors

2
Issues

143
Stars

55
Forks



lehome-challenge/docs/training.md at main • lehome-official/lehome-challenge

evaluate

1-2.

Register Policy

Allow the evaluation system to
find policy by name

1-1.

Create Policy Class

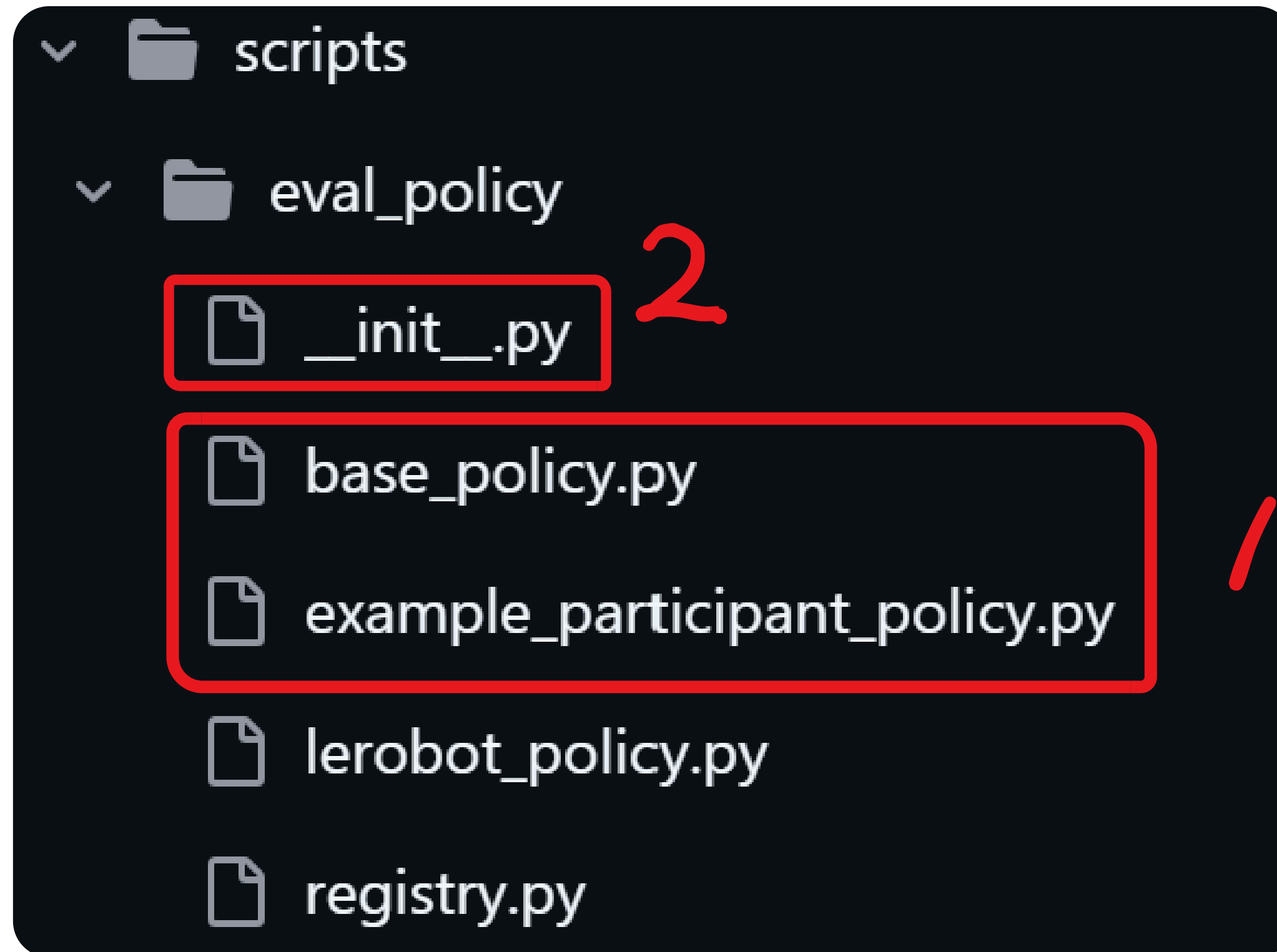
Package the model into a unified interface
that can be called by the evaluation system

2.

Import in init.py

Make sure Python
actually loads your
module at runtime

evaluate



1-1, 1-2

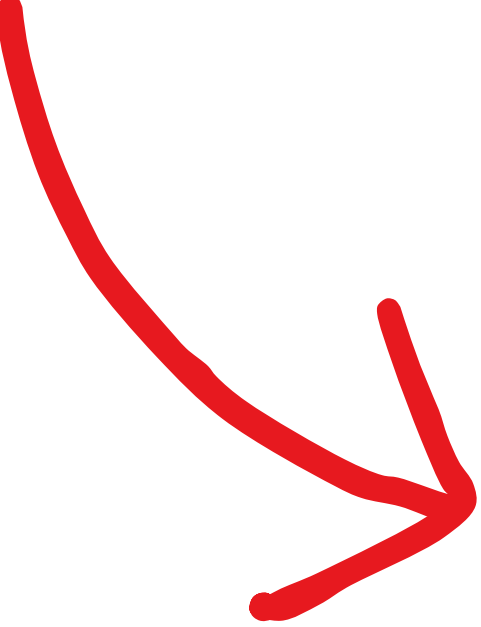
evaluate

1-1.

Create Policy Class

1-2.

Register Policy



```
# my_policy.py
import numpy as np
from typing import Dict
from scripts.eval_policy.base_policy import BasePolicy
from scripts.eval_policy.registry import PolicyRegistry

@PolicyRegistry.register("my_policy")
class MyPolicy(BasePolicy):
    def __init__(self, model_path=None, device="cuda", **kwargs):
        super().__init__(**kwargs)
        self.device = device
        # Load your model here

    def reset(self):
        """Called at the start of each episode."""
        pass

    def select_action(self, observation: Dict[str, np.ndarray]) -> np.ndarray:
        """
        Args:
```


evaluate

2. Import in init.py

Make sure Python actually loads your module at runtime

```
# scripts/eval_policy/__init__.py  
from .my_policy import MyPolicy
```

evaluate

Evaluate Our Policy

```
python -m scripts.eval \  
    --policy_type custom \  
    --policy_path "path/to/your/model" \  
    --garment_type "tops_long" \  
    --num_episodes 5 \  
    --enable_cameras \  
    --device cpu
```